EXP 2	A PYTHON PROGRAM TO SOLVE THE TRANSPORTATION PROBLEM

AIM:

To solve the Transportation Problem for minimizing the cost of transporting goods from multiple sources to multiple destinations using Python.

PROCEDURE:

1. Install Required Libraries:

o Make sure you have numpy and scipy installed. You can install them using pip if

not already installed.

bash

Copy code

pip install numpy scipy

2. Define the Problem:

o Identify the cost matrix, supply array, and demand array.

3. Set Up the Problem in Python:

o Use the scipy.optimize.linprog function to set up and solve the Transportation

Problem.

4. Run the Code:

o Execute the Python code to get the optimized result.

5. Analyze the Output:

o Interpret the results and validate the solution against the problem constraints.

PROBLEM 1: SOLVE THE IFBS FOR THE PROBLEM

	D1	D2	D3	SUPPLY
S1	4	8	8	50
S2	2	7	6	40
S3	3	4	2	60
DEMAND	30	70	50	

CODE:

```
import numpy as np
from scipy.optimize import linprog
def transportation_problem(supply, demand, costs):
  supply = list(supply)
  demand = list(demand)
  if sum(supply) != sum(demand):
    if sum(supply) < sum(demand):
      diff = sum(demand) - sum(supply)
      supply.append(diff)
      costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))
    else:
      diff = sum(supply) - sum(demand)
      demand.append(diff)
      costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))
  num_sources = len(supply)
  num_destinations = len(demand)
  A_eq_supply = np.zeros((num_sources, num_sources *
num_destinations))
  for i in range(num_sources):
    A_eq_supply[i, i * num_destinations:(i + 1) * num_destinations] = 1
```

```
  A_eq_demand = np.zeros((num_destinations, num_sources *
num_destinations))
  for j in range(num_destinations):
    A_eq_demand[j, j::num_destinations] = 1
  A_eq = np.vstack((A_eq_supply, A_eq_demand))
  b_eq = np.array(supply + demand)
  c = costs.flatten()
  bounds = (0, None)
  result = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
method='highs')
  if result.success:
    allocation = result.x.reshape(num_sources, num_destinations)
    total_cost = result.fun
    return allocation, total_cost
  else:
    print("Optimization failed:", result.message)
    return None, None
supply = [50, 40, 60]
demand = [30, 70, 50]
costs = np.array([[4, 8, 8], [2, 7, 6], [3, 4, 2]])
allocation, total_cost = transportation_problem(supply, demand, costs)
if allocation is not None:
  print("Optimal Allocation:")
  print(allocation)
  print("Total Cost:", total_cost)
```

EXPECTED OUTPUT:

Optimal Allocation:

```
[[ 0. 50.  0.]
 [30. 10.  0.]
 [ 0. 10. 50.]]
Total Cost: 670.0
```

PROBLEM 2: CONSIDER THE FOLLOWING TRANSPORTATION PROBLEM INVOLVING 3 SOURCES AND 4 DESTINATIONS.

	D1	D2	D3	D4	SUPPLY
S1	3	1	7	4	300
S2	2	6	5	9	150
S3	8	3	3	2	500
DEMAND	250	150	400	200	

CODE:

```
import numpy as np
from scipy.optimize import linprog

# Supply and Demand
supply = [300, 150, 500]
demand = [250, 150, 400, 200]

# Cost matrix
costs = np.array([
    [3, 1, 7, 4],
    [2, 6, 5, 9],
    [8, 3, 3, 2]
])
```

```
# Balance the problem
if sum(supply) < sum(demand):
    diff = sum(demand) - sum(supply)
    supply.append(diff)
    costs = np.vstack([costs, np.zeros((1, costs.shape[1]))])
elif sum(supply) > sum(demand):
    diff = sum(supply) - sum(demand)
    demand.append(diff)
    costs = np.hstack([costs, np.zeros((costs.shape[0], 1))])

# Updated sizes
m = len(supply)
n = len(demand)
# Objective function
c = costs.flatten()
# Equality constraints
A_eq = []
b_eq = []
# Supply constraints
for i in range(m):
    row = np.zeros(m * n)
    for j in range(n):
        row[i * n + j] = 1
    A_eq.append(row)
    b_eq.append(supply[i])
# Demand constraints
for j in range(n):
```

```
    row = np.zeros(m * n)
    for i in range(m):
        row[i * n + j] = 1
    A_eq.append(row)
    b_eq.append(demand[j])
A_eq = np.array(A_eq)
b_eq = np.array(b_eq)
# Solve using HiGHS method
result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')
# Output
allocation = result.x.reshape((m, n))
print("Optimal Allocation:")
print(allocation)
print("\nMinimum Transportation Cost:", result.fun)
```

OUTPUT:

Optimal Allocation Matrix:

```
[[  0. 150.   0. 150.]
 [150.   0.   0.   0.]
 [ 50.   0. 400.  50.]
 [ 50.   0.   0.   0.]]
```

RESULT:

Thus The Transportation Problem For Minimizing The Cost Of Transporting Goods From Multiple Sources To Multiple Destinations Using Python Was Executed Successfully.